```
"""
Gold + Semiconductor trend-following bot (Alpaca) with Telegram alerts to your iP
Defaults to PAPER trading. Run once per day near market close (cron / cloud sched

Setup:
  pip install alpaca-py pandas requests
  export ALPACA_KEY=...  ALPACA_SECRET=...
  export TG_TOKEN=...    TG_CHAT_ID=...        # Telegram bot (optional alerts)
  export PAPER=true      DRY_RUN=true          # flip to false only after testing
"""
import os
import datetime as dt
import requests
from alpaca.trading.client import TradingClient
from alpaca.trading.requests import MarketOrderRequest
from alpaca.trading.enums import OrderSide, TimeInForce
from alpaca.data.historical import StockHistoricalDataClient
from alpaca.data.requests import StockBarsRequest
from alpaca.data.timeframe import TimeFrame
from alpaca.data.enums import DataFeed

GOLD = ["GLD", "GDX"]                       # gold ETF + gold miners
CHIPS = ["SMH", "NVDA", "AMD", "AVGO"]      # semiconductor ETF + big chip names
SYMBOLS = GOLD + CHIPS

FAST, SLOW = 20, 50        # moving-average windows (days)
POSITION_PCT = 0.08        # 8% of equity per symbol
MAX_INVESTED_PCT = 0.60    # never hold more than 60% of equity
STOP_LOSS = -0.08          # close a position at -8%

PAPER = os.getenv("PAPER", "true").lower() == "true"
DRY_RUN = os.getenv("DRY_RUN", "true").lower() == "true"

trading = TradingClient(os.environ["ALPACA_KEY"], os.environ["ALPACA_SECRET"], pa
data = StockHistoricalDataClient(os.environ["ALPACA_KEY"], os.environ["ALPACA_SEC


def notify(msg: str) -> None:
    print(msg)
    token, chat = os.getenv("TG_TOKEN"), os.getenv("TG_CHAT_ID")
    if token and chat:
        try:
            requests.post(
```

```
                f"https://api.telegram.org/bot{token}/sendMessage",
                json={"chat_id": chat, "text": msg},
                timeout=10,
            )
        except requests.RequestException as e:
            print("Telegram error:", e)


def place(symbol: str, side: OrderSide, notional: float = None) -> None:
    if DRY_RUN:
        notify(f"[DRY RUN] {side.value} {symbol}")
        return
    if side == OrderSide.SELL:
        trading.close_position(symbol)
    else:
        trading.submit_order(
            MarketOrderRequest(
                symbol=symbol, notional=round(notional, 2),
                side=side, time_in_force=TimeInForce.DAY,
            )
        )
    notify(f"{'PAPER ' if PAPER else ''}{side.value.upper()} {symbol}")


def main() -> None:
    if not trading.get_clock().is_open:
        print("Market closed, skipping.")
        return

    start = dt.datetime.now(dt.timezone.utc) - dt.timedelta(days=150)
    bars = data.get_stock_bars(
        StockBarsRequest(symbol_or_symbols=SYMBOLS, timeframe=TimeFrame.Day,
                         start=start, feed=DataFeed.IEX)
    ).df

    equity = float(trading.get_account().equity)
    positions = {p.symbol: p for p in trading.get_all_positions()}
    invested = sum(float(p.market_value) for p in positions.values())

    for sym in SYMBOLS:
        if sym not in bars.index.get_level_values(0):
            continue
        close = bars.loc[sym]["close"]
        if len(close) < SLOW:
            continue
        fast, slow, price = close.tail(FAST).mean(), close.tail(SLOW).mean(), clo
```

```
        held = positions.get(sym)

        if held:
            pnl = float(held.unrealized_plpc)
            if pnl <= STOP_LOSS:
                place(sym, OrderSide.SELL)
                notify(f"Stop-loss hit on {sym} ({pnl:.1%})")
            elif fast < slow or price < slow:
                place(sym, OrderSide.SELL)
        elif fast > slow and price > slow:
            size = equity * POSITION_PCT
            if invested + size <= equity * MAX_INVESTED_PCT:
                place(sym, OrderSide.BUY, size)
                invested += size

    notify(f"Run complete. Equity: ${equity:,.0f}")


if __name__ == "__main__":
    main()
```